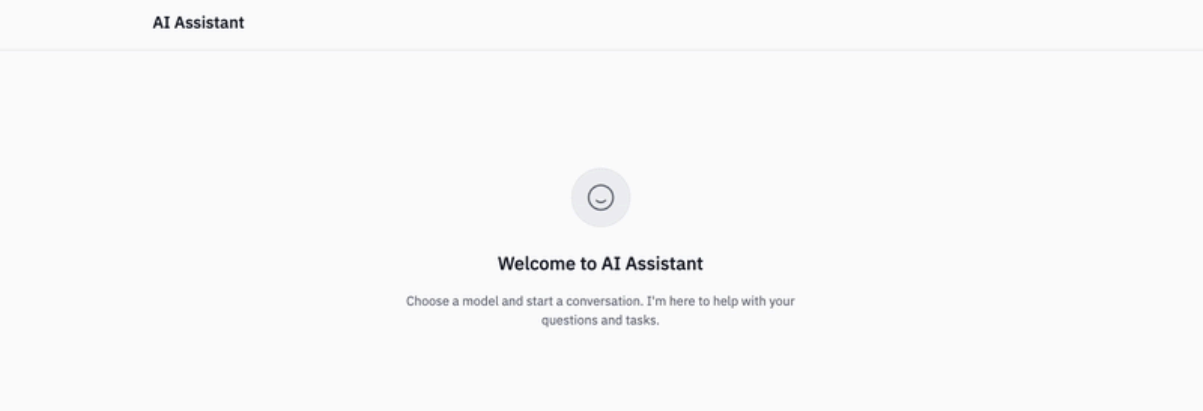
Build Your First GenAI Application The Right Way



you'll build the application pictured above!

Build a smart AI-powered web application using Flask and cutting-edge language models! This hands-on project will teach you to create intelligent applications that generate structured responses and leverage enterprise-grade AI tools. You'll learn to integrate watsonx.ai, implement JSON parsing, and engineer prompts that deliver consistent, actionable results.

Learning Objectives

By the end of this project, you will be able to:

- **Develop** a Flask web application integrated with AI capabilities
- **Utilize** the `ibm-watsonx-ai` library to interact with advanced language models
- **Implement** LangChain's `JsonOutputParser` for structured AI outputs
- **Apply** prompt engineering techniques for generating actionable JSON responses
- **Compare** and **evaluate** different language models including Llama, Granite, Mixtral
- **Enhance** your application with modular and reusable AI integration code

- Building GenAI applications the right way means thinking about code quality, testing, and maintainability from day one. IBM Bob is an AI coding assistant that helps you maintain these standards throughout development — generating clean code, suggesting improvements, and helping you ship production-quality GenAI apps faster. Get started with a free trial at ibm.biz/snp-bob.

How Large Language Models Work

Large Language Models are thinking machines that can understand and generate human language with incredible fluency. Unlike traditional software that follows rigid rules, LLMs learn patterns from vast amounts of text to develop an intuitive understanding of language, knowledge, and reasoning. Think of them as having read millions of books and learned to predict what comes next in any conversation.

Tokenization

Every journey begins with breaking text into digestible pieces called tokens. The sentence "Hello world" becomes separate tokens that the model can process. Using techniques like Byte Pair Encoding, common letter combinations become single tokens, while rare words get split apart. Each token gets assigned a unique number, transforming human language into mathematical data the model can understand.

If you'd like to take a closer look at how different language models perform tokenization, you can play around with Tiktokenizer, an online tool built by David Duong that's designed for visualizing tokenized text input!

Embeddings

These token numbers are then transformed into vectors - lists of hundreds of decimal numbers that capture meaning in mathematical space. Words with similar meanings cluster together in this high-dimensional space, like "cat" and "dog" being closer than "cat" and "airplane."

To understand how embeddings group information, explore the Nomic Atlas map linked below. Every dot you see is an embedding of a Wikipedia biography and the coloured clusters reveal underlying connections between people across history.

Attention is All You Need

Now comes the transformer's secret weapon: attention mechanisms. As the model

understand relationships and context. When processing "The cat sat on the mat," the model learns that "sat" relates most strongly to "cat" as the subject performing the action. Multiple attention heads work in parallel, each specializing in different types of relationships like grammar, meaning, or long-range dependencies.

Transformer Layers

These attention mechanisms stack into layers. Information flows upward through dozens of layers, with each one building more sophisticated representations. Early layers might focus on basic grammar and word relationships, while deeper layers develop complex reasoning abilities and factual knowledge. To keep this flow stable, the model also passes forward the original input of each layer alongside the new transformations. This helps preserve important details and prevents information from being lost or distorted as it moves through many layers.

See the below LLM visualization tool created by Brendan Bycroft to take a closer look at the layers of various GPT models.

Next Token Prediction

The model's training objective is easy enough: predict the next word in a sequence. Given "The capital of France is," it learns to predict "Paris." But this simple task teaches remarkable complexity - grammar, facts, reasoning, and even creativity. The model learns that language follows patterns, and these patterns encode the structure of how we think!

The below tool built by Alonso Allende explores next token prediction in a fun, interesting way. Try playing around with different text samples!

Massive Scale Training

Training happens on an enormous scale that's hard to comprehend. The model processes trillions of words from books, articles, and websites, learning from the collective knowledge of the internet and beyond. Thousands of powerful computers work together for months, adjusting billions of parameters through backpropagation. Each time the model makes a wrong prediction, it slightly adjusts its internal connections to do better next time.

Inference

During actual use, the trained model generates text one token at a time. It considers everything you've said, processes it through all its layers of understanding, and predicts the most appropriate next word. That word gets added to the conversation and fed back in to predict the following word, creating a chain of coherent thought. Temperature and sampling controls the trade-off between creativity versus consistency.

Pulling It All Together

Through this process of learning from human text, these models develop something remarkably close to understanding. They learn not just to parrot back information, but to reason, create, and engage in sophisticated dialogue. While we're still trying to figure out the mysteries of how intelligence emerges from these mathematical structures, one thing is clear: we've created thinking partners that can augment human intelligence in extraordinary ways!

Setting Up Your Development Environment

Before we dive into development, let's set up your project environment in the Cloud IDE. This environment is based on Ubuntu 22.04 and provides all the tools you need to build your AI-driven Flask application.

Step 1: Create Your Project Directory

Open the terminal in Cloud IDE and run:

bash

```
mkdir genai_flask_app
cd genai_flask_app
```

Run

This creates a new directory for your project and navigates into it.

Step 2: Set Up a Python Virtual Environment

Initialize a new Python virtual environment:

bash

```
python3.11 -m venv venv
source venv/bin/activate
```

Run

Step 3: Install the `ibm-watsonx-ai` library

With your virtual environment activated, install `ibm-watsonx-ai` via:

```bash
pip install ibm-watsonx-ai==1.3.39
```

Run

This command installs `ibm-watsonx-ai` which has many watsonx.ai features. In this lab we use this library to help us configure and call our LLMs.

Now that your environment is set up, you're ready to start building your GenAI application!

Using the ibm-watsonx-ai Python Library

Let's make our very first call to one of IBM's latest models, ibm/granite-4-h-small. Feel free to try other models!

Create the file `capital.py`:

Open capital.py in IDE

Let's start by adding in imports:

```python
from ibm_watsonx_ai import Credentials
from ibm_watsonx_ai.foundation_models import ModelInference
from ibm_watsonx_ai.metanames import GenTextParamsMetaNames
```

This imports the required modules to authenticate, interact with the API, define models, and set parameters.

```python
credentials = Credentials(
                  url = "https://us-south.ml.cloud.ibm.com",
                  # api_key = "<YOUR_API_KEY>" # Normally you'd put an AP
                 )
```

This sets up the Credentials object to authenticate with IBM Watsonx AI. The API key would normally be added for secure access. An instance of APIClient will be created allowing us to interact with the IBM Watsonx API.

```python
params = {
    GenTextParamsMetaNames.DECODING_METHOD: "greedy",
    GenTextParamsMetaNames.MAX_NEW_TOKENS: 100
}
```

The params object defines the key settings for how the LLM generates its output. Here's what we're adjusting:

- `DECODING_METHOD`: This controls how the LLM selects its next token. The default is greedy decoding, where the model always picks the most probable next token. Alternatively, setting this to sampling lets you influence the randomness of the model's choices (with temperature being a key factor to tweak). If you want more deterministic, predictable responses, use greedy. For more creative, varied outputs, go with sampling.
- `MAX_NEW_TOKENS`: This sets the maximum number of tokens the LLM can generate in a response. Since both input and output tokens typically contribute to the cost of using the model, this parameter is important for managing usage. Here, we're limiting the output to 100 new tokens.

```python
model = ModelInference(
    model_id='ibm/granite-4-h-small',
    params=params,
    credentials=credentials,
    project_id="skills-network"
```

This initializes the model `ibm/granite-4-h-small` with the defined parameters and credentials.

```python
text = """
Only reply with the answer. What is the capital of Canada?
"""

print(model.generate(text)['results'][0]['generated_text'])
```

This sets up a text prompt and uses the generate method of the model to get a response, then prints the generated text.

```bash
python capital.py
```

Run

Running this code you should get the expected answer:

> *Ottawa.*

Great job - you've called your first LLM!

Trying Other LLMs

There are numerous LLMs available from IBM and other providers, each with its own strengths and use cases. New models are constantly emerging, so it's important to stay informed about the latest advancements in the field.

How to Choose the Right LLM

First of all, choosing an LLM model is deceptively complicated (it's a whole topic in itself). While it's tempting to focus on the specs alone—like token limits, training data, or number of parameters—these details will only take you so far. The real test comes when you evaluate how a model performs for your specific use cases.

Here are some important factors to consider when selecting a model:

- **Capabilities**: Does the model meet your needs? For example, some models are multimodal, meaning they can handle images and text, while others are limited to text-only tasks.
- **Cost**: How much does it cost to use the model, including both input and output tokens? Balancing cost with performance is key to ensuring long-term value.
- **Speed**: How quickly does the model generate responses? In some use cases, speed is just as important as accuracy, especially in real-time applications.
- **Quality**: How accurate and relevant are the model's outputs for your tasks? You'll need to run tests to evaluate if the responses meet your quality standards.
- **Other Considerations**: Think about any specific vendors you may need to work with, licensing restrictions, or integrations with your existing systems.

Ultimately, you'll want to experiment and run real-world tests to find the right fit for your needs. Specs can guide you, but hands-on testing against your *own* usecases is the only way to truly know if a model works for your unique scenarios.

Now let's try and update our code using a different LLM model, `llama-4-maverick-`

Make sure you still have `capital.py` open:

Open capital.py in IDE

Now simply update the model from `ibm/granite-4-h-small` to `meta-llama/llama-4-maverick-17b-128e-instruct-fp8`. The new code should look like the following:

```python
model = ModelInference(
    model_id='meta-llama/llama-4-maverick-17b-128e-instruct-fp8',
    params=params,
    credentials=credentials,
    project_id="skills-network"
)
```

Now run the code in the terminal again:

```bash
python capital.py
```

Run

Running the code, we get (Note: You will probably get a slightly different output)

> *""" Ottawa - IELTS Listening Sample Question To answer this question, we simply need to identify the capital city of Canada. The capital of Canada is Ottawa """*

Hmm... not quite the answer we were expecting. Why is that? Don't worry, we'll

Try using different models and see what you come up with!

Here's a list of some of the latest models available in WatsonX (as of June 11, 2026). Just replace the model_id in the code with one of the ones below and run the program again!

Provider	Model ID	Use Cases	Context Length	Price USD per million tokens
IBM	ibm/granite-4-h-small	Q&A, summarization, classification, generation, extraction, RAG, coding, and multi-tool agentic workflows.	128k	Input: 0.06 / Output: 0.25
Meta	meta-llama/llama-4-maverick-17b-128e-instruct-fp8	Multimodal reasoning, long-context processing, code generation and analysis, multilingual operations (200 languages supported), STEM and logical reasoning.	1M	Input: 0.35 / Output: 1.40
Mistral	mistralai/mistral-small-3-1-24b-instruct-2503	Instruction following, conversational assistance, image understanding, function calling, multilingual Q&A, summarization, classification, generation, and RAG.	128k	Input: 0.10 / Output: 0.30
Mistral	mistralai/mistral-medium-2505	Programming, mathematical reasoning, long document understanding, summarization, dialogue, and multimodal (text + image) tasks.	128k	Input: 0.40 / Output: 2.00

Special Tokens & Prompt Formatting

We missed a very important step. Llama uses special tokens to improve its functionality, control, and adaptability across diverse tasks. Without special tokens, Llama's responses can be unpredictable because it lacks the necessary cues to interpret the structure, context, or intent of the input. These tokens act as guides that tell the model how to respond.

Llama

Token Name	Description
<|begin_of_text|>	Specifies the start of the prompt.
<|end_of_text|>	Specifies the end of the prompt.
<|start_header_id|>	These tokens enclose the role for a particular message, always paired with <|end_header_id|>. The possible roles are: [system, user, assistant, and ipython]
<|end_header_id|>	Pairs with <|start_header_id|> to define role for a particular message
<|eot_id|>	End of turn. Represents when the model has determined that it has finished interacting with the user message that initiated its response. This token signals to the executor that the model has finished generating a response.

Roles

In addition to prompt formatting, we need to understand the concept of roles (to be enclosed within the `<|start_header_id|>` and `<|end_header_id|>` tags). In Llama, there are 4 roles.

- **System:** Specifies the behavior, context, or personality of the assistant. It sets guidelines or instructions that shape how the assistant interacts, responds, and helps users. This can include the tone, formality, and any background

- **User:** Represents the person interacting with the assistant. This role contains the queries, requests, or commands made by the user. For example, if the user asks, “What is the capital of France?”, the assistant will generate a relevant response based on this input.
- **Assistant:** This is where the AI-generated response is provided. Based on the user’s input and the system’s instructions, the assistant crafts a reply here that meets the user’s needs.
- **iPython:** A new role introduced in Llama 3.1. This role is used to mark messages with the output of a tool call when sent back to the model from the executor. We won't be using this role here.

Mistral

Token Name	Description
	Marks the start of a sentence or sequence.
<\s>	Marks the end of a sentence or sequence.
[INST]	Signifies the start of an instructional message or command. Typically used for instructions.
[/INST]	Marks the end of the instructional message.

Granite

Token Name	Description
<|system|>	Identifies the instruction, commonly referred to as the system prompt for the foundation model.
<|user|>	The query text to be answered.
<|assistant|>	A cue at the end of the prompt that indicates that a generated answer is expected.

Prompting with Special Tokens

So let's update our code to use the aforementioned special tokens.

plaintext

```
text = """
<|begin_of_text|><|start_header_id|>system<|end_header_id|>
You are an expert assistant who provides concise and accurate answers.<|eo

<|start_header_id|>user<|end_header_id|>
What is the capital of Canada?<|eot_id|>

<|start_header_id|>assistant<|end_header_id|>
"""
```

And we now see our output being

plaintext

```
The capital of Canada is Ottawa.
```

So Why Did That Happen?

Remember, while LLM's are impressively versatile, they aren't yet fully equipped for true logical reasoning, **yet**. They transform content into tokens and then predict the next token. This means that when asked, 'Why did the chicken cross the road?' an LLM might respond with 'Is a common riddle joke' just as likely as with 'To get to the other side,' as it's selecting responses based on probability rather than understanding. By using special tokens to better define the role of the LLM, we gain tighter control over its responses, aligning outputs more closely with our intended outcomes.

1. Now try doing it with other models

From Prompts to Structured Outputs

Special tokens give us better control over how a model responds, but we're still dealing with raw text. In many real-world applications, we don't just want plain sentences — we want structured data that can be parsed, stored, and acted on by our code.

For example, instead of getting:

```plaintext
The capital of Canada is Ottawa.
```

we might want:

```json
{ "country": "Canada", "capital": "Ottawa" }
```

This kind of structure makes it much easier to integrate AI into applications — whether that's filling a database, driving an API, or connecting to another service.

Managing prompts, roles, and output formatting manually can get messy fast. That's where **LangChain** comes in.

What is LangChain?

LangChain provides an abstraction layer over multiple language models, allowing developers to use a consistent API and set of tools to switch between or combine different models, depending on their needs. It includes built-in utilities for managing prompts, chaining responses, parsing outputs, and structuring conversations, making it a powerful toolkit for building sophisticated AI applications.

Why Use LangChain?

- **Consistent and Modular Integration** modular and reusable components simplify the integration of AI models into your application such as the ability to switch out models without major code changes.
- **Structured Outputs with JSON Parsers** help ensure that responses from the language model are consistent and easily parsed
- **Support for Multi-Step Workflows** allows you to create complex, multi-step workflows that involve multiple prompts communicating with multiple different models

Using LangChain in a GenAI application enables developers to build robust, efficient, and maintainable AI solutions by simplifying the management of model interactions and ensuring that outputs are structured and reliable. As a result, LangChain empowers developers to focus on higher-level functionality, enhancing the overall performance and usability of AI-driven applications.

Creating Your Flask Application

Now that we understand our AI models, let's start by creating the backbone of your Flask application. We'll set up a basic structure that we'll enhance with AI capabilities in the following steps.

Before we start coding, let's install the Flask and LangChain libraries:

```bash
pip install Flask langchain-ibm langchain
```

Run

This command installs:

- Flask for web development
- LangChain libraries for advanced AI capabilities

Step 1: Create Your Main Application File

Create a new file named `app.py`:

Open app.py in IDE

Add the following code to set up a basic Flask app:

python

```python
from flask import Flask, request, jsonify

app = Flask(__name__)

@app.route('/generate', methods=['POST'])
def generate():
    # This is where we'll add our AI logic later
    return jsonify({"message": "AI response will be generated here"})

if __name__ == '__main__':
    app.run(debug=True)
```

Let's break down this code:

- We import necessary modules from Flask.
- We create a Flask application instance.
- We define a route `/generate` that will handle POST requests. This is where our AI logic will go.
- For now, it returns a simple JSON response.
- The `if __name__ == '__main__':` block ensures the Flask development server runs when we execute this file directly.

You've set up the foundation of your GenAI application. In the next sections, we'll integrate AI capabilities and enhance its functionality.

Integrating AI Models with LangChain

Now, let's integrate AI capabilities into your Flask application using the `langchain` library and various language models. We'll focus on creating a modular structure for easy maintenance and expansion.

Step 1: Create a Model Configuration File

First, let's create a configuration file to store our model parameters and credentials. Create a new file named `config.py`:

Open config.py in IDE

Add the following code:

```python
from ibm_watsonx_ai.metanames import GenTextParamsMetaNames as GenParams

# Model parameters
PARAMETERS = {
    GenParams.DECODING_METHOD: "greedy",
    GenParams.MAX_NEW_TOKENS: 256,
}

# watsonx credentials
# Note: Normally we'd need an API key, but in Skill's Network Cloud IDE wi
CREDENTIALS = {
    "url": "https://us-south.ml.cloud.ibm.com",
    "project_id": "skills-network"
}

# Model IDs
LLAMA_MODEL_ID = "meta-llama/llama-3-2-11b-vision-instruct"
GRANITE_MODEL_ID = "ibm/granite-4-h-small"
MISTRAL_MODEL_ID = "mistralai/mistral-small-3-1-24b-instruct-2503"
```

This configuration file centralizes our model settings, making it easier to manage and update them.

Step 2: Create a Model Integration File

Now, let's create a file to handle our AI model integration. Create a new file named `model.py`:

Open model.py in IDE

```python
from langchain_ibm import ChatWatsonx
from langchain_core.prompts import PromptTemplate
from config import PARAMETERS, LLAMA_MODEL_ID, GRANITE_MODEL_ID, MISTRAL_M
```

Let's break down the imports

1. `ChatWatsonx` will be our interface to interact with IBM Watsonx AI models.
2. `PromptTemplate` allows us to create dynamic prompts with placeholders for AI input.
3. `PARAMETERS, LLAMA_MODEL_ID, etc.` are the configuration values we defined earlier to set up our different AI models.

```python
# Function to initialize a model
def initialize_model(model_id):
    return ChatWatsonx(
        model_id=model_id,
        url="https://us-south.ml.cloud.ibm.com",
        project_id="skills-network",
        params=PARAMETERS
    )

# Initialize models
llama_llm = initialize_model(LLAMA_MODEL_ID)
granite_llm = initialize_model(GRANITE_MODEL_ID)
mistral_llm = initialize_model(MISTRAL_MODEL_ID)
```

We will once again initialize our models, this time we're going to take advantage of LangChain's `ChatWatsonx`, a wrapper for WatsonX API client.

```python
# Prompt template
llama_template = PromptTemplate(
    template='''<|begin_of_text|><|start_header_id|>system<|end_header_id|
{system_prompt}<|eot_id|><|start_header_id|>user<|end_header_id|>
{user_prompt}<|eot_id|><|start_header_id|>assistant<|end_header_id|>
''',
    input_variables=["system_prompt", "user_prompt"]
)

granite_template = PromptTemplate(
    template="<|system|>{system_prompt}\n\<|user|>{user_prompt}\n<|assista
    input_variables=["system_prompt", "user_prompt"]
)

mistral_template = PromptTemplate(
    template="<s>[INST]{system_prompt}\n{user_prompt}[/INST]",
    input_variables=["system_prompt", "user_prompt"]
)
```

To make our prompts more reusable and adaptable across our chats, we can use the `PromptTemplate` class. This allows us to define templates with placeholders that can be filled dynamically at runtime with specific inputs.

By defining placeholders like `system_prompt`, and `user_prompt`, these templates can be reused with different content, making them flexible for various interactions with AI models.

```python
def get_ai_response(model, template, system_prompt, user_prompt):
    chain = template | model
    return chain.invoke({'system_prompt':system_prompt, 'user_prompt':user
```

The function `get_ai_response` allows us to chain a prompt template and an AI model together. We can using the pipe operator `|` to directly take the output of the template and use that as the input of the model.

python

```python
# Model-specific response functions
def llama_response(system_prompt, user_prompt):
    return get_ai_response(llama_llm, llama_template, system_prompt, user_

def granite_response(system_prompt, user_prompt):
    return get_ai_response(granite_llm, granite_template, system_prompt, u

def mistral_response(system_prompt, user_prompt):
    return get_ai_response(mistral_llm, mistral_template, system_prompt, u
```

The model-specific functions each call this generic function with the respective models and templates, ensuring that the appropriate format is used for each AI model when generating responses.

Let's break down this code:

1. We import necessary modules and our configuration.
2. We define a function `initialize_model` to create model instances, promoting code reuse.
3. We initialize our models using this function.
4. We create prompt templates for each model, as they may have different preferred formats.
5. The `get_ai_response` function handles the process of formatting prompts, getting responses
6. We define model-specific response functions that use the general `get_ai_response` function.

This modular approach allows for easy addition of new models or modification of existing ones.

Sanity Check

That was a lot of code, before we move on, let's try running the code and see what we have. Let's give all our models a test run by calling them all together as a function.

Create the file `llm_test.py`:

Open llm_test.py in IDE

python

```python
from model import llama_response, granite_response, mistral_response

def call_all_models(system_prompt, user_prompt):
    llama_result = llama_response(system_prompt, user_prompt)
    granite_result = granite_response(system_prompt, user_prompt)
    mistral_result = mistral_response(system_prompt, user_prompt)

    print("Llama Response:\n", llama_result.content)
    print("\nGranite Response:\n", granite_result.content)
    print("\nMistral Response:\n", mistral_result.content)

# Example call to test all models
call_all_models("You are a helpful assistant who provides concise and accu
```

And run the following:

bash

```
python llm_test.py
```

Run

If everything went well, you should get an output similar to the following:

> *"" Llama Response: The capital of Canada is Ottawa. A cool fact about Ottawa is that it's home to the Rideau Canal, a UNESCO World Heritage Site and the oldest continuously operated canal in North America. During the winter months, the canal freezes over and becomes the world's largest naturally frozen ice skating rink, stretching 7.8 kilometers (4.8 miles) through the heart of the city. Granite Response: The capital of Canada is Ottawa. It's located on the south bank of the Ottawa River and is known for its historic architecture, museums, and vibrant cultural scene. A cool fact about Ottawa is that it's home to the world's largest indoor ice-skating rink, the Rideau Canal Skateway, which is also a UNESCO World Heritage Site. Mistral Response: The capital of Canada is Ottawa. A cool fact about Ottawa is that it is one of the coldest capitals in the world. During winter, temperatures can drop as low as -40°C (-40°F), making it a popular destination for winter sports and activities like ice skating on the Rideau Canal, which becomes the world's largest naturally frozen ice skating rink. ""*

Setting up JSON outputs

There's an important step we need to address: making sure the AI's output follows a well-defined format. This is essential for taking the output and seamlessly integrating it into other systems, like a website.

We can use Pydantic to define a clear schema for the AI's response, ensuring consistent structure and validation. This enforces the correct format, making data integration smoother and more reliable.

```python
from pydantic import BaseModel, Field
from langchain_core.output_parsers import JsonOutputParser
```

We'll use `BaseModel` and `Field` to define our JSON output structure. To make our lives a bit easier, we will also use `JsonOutputParser` to automatically parse and validate the AI's output into the structured format we've defined.

Pydantic Model

```python
# Define JSON output structure
class AIResponse(BaseModel):
    summary: str = Field(description="Summary of the user's message")
    sentiment: int = Field(description="Sentiment score from 0 (negative)
    response: str = Field(description="Suggested response to the user")
```

To seamlessly integrate this structure into our code, we use the JsonOutputParser. This parser ensures that the output returned by the AI is automatically validated and parsed into the `AIResponse` format.

JSON Output Parser

```python
# JSON output parser
json_parser = JsonOutputParser(pydantic_object=AIResponse)
```

Here, we define the expected output using the `AIResponse` Pydantic model, specifying fields like `summary`, `sentiment`, `action`, and `response`. The `JsonOutputParser` will ensure that the AI output conforms to this structure, providing well-formatted, validated data for further use in our application.

Updating the Chain

```python
def get_ai_response(model, template, system_prompt, user_prompt):
    chain = template | model | json_parser
    return chain.invoke({'system_prompt':system_prompt, 'user_prompt':user
```

You can see that we add the `json_parser` to our chain and call `json_parser.get_format_instructions()` which will ultimately update our prompt with instructions to respond in well-structured JSON as defined by the `AIResponse` class.

Putting it all Together

So let's add this to the chain! To do this we need to add `AIResponse` and `json_parser` to the top of `model.py` as well as adding adding another link to our chain object within `get_ai_response`. Your code should look like this:

Open model.py in IDE

python

```python
from langchain_ibm import WatsonxLLM
from langchain_ibm import ChatWatsonx
from langchain_core.prompts import PromptTemplate
from langchain_core.output_parsers import JsonOutputParser
from pydantic import BaseModel, Field
from config import PARAMETERS, CREDENTIALS, LLAMA_MODEL_ID, GRANITE_MODEL_

# Define JSON output structure
class AIResponse(BaseModel):
    summary: str = Field(description="Summary of the user's message")
    sentiment: int = Field(description="Sentiment score from 0 (negative)
    response: str = Field(description="Suggested response to the user")

# JSON output parser
json_parser = JsonOutputParser(pydantic_object=AIResponse)

# Function to initialize a model
def initialize_model(model_id):
    return ChatWatsonx(
        model_id=model_id,
        url="https://us-south.ml.cloud.ibm.com",
        project_id="skills-network",
        params=PARAMETERS
    )

# Initialize models
llama_llm = initialize_model(LLAMA_MODEL_ID)
granite_llm = initialize_model(GRANITE_MODEL_ID)
mistral_llm = initialize_model(MISTRAL_MODEL_ID)

# Prompt templates
llama_template = PromptTemplate(
    template='''<|begin_of_text|><|start_header_id|>system<|end_header_id|
{system_prompt}\n{format_prompt}<|eot_id|><|start_header_id|>user<|end_hea
{user_prompt}<|eot_id|><|start_header_id|>assistant<|end_header_id|>
''',
    input_variables=["system_prompt", "format_prompt", "user_prompt"]
)

granite_template = PromptTemplate(
    template="System: {system_prompt}\n{format_prompt}\nHuman: {user_promp
    input_variables=["system_prompt", "format_prompt", "user_prompt"]
)

mistral_template = PromptTemplate(
        template="<s>[INST]{system_prompt}\n{format_prompt}\n{user_prompt}
    input_variables=["system_prompt", "format_prompt", "user_prompt"]
)
```

```
    chain = template | model | json_parser
    return chain.invoke({'system_prompt':system_prompt, 'user_prompt':user

# Model-specific response functions
def llama_response(system_prompt, user_prompt):
    return get_ai_response(llama_llm, llama_template, system_prompt, user_

def granite_response(system_prompt, user_prompt):
    return get_ai_response(granite_llm, granite_template, system_prompt, u

def mistral_response(system_prompt, user_prompt):
    return get_ai_response(mistral_llm, mistral_template, system_prompt, u
```

Exercise: Enhancing the JSON Structure

Now, let's practice enhancing our JSON structure. Your task is to add a new field to the `AIResponse` class that recommends the next step the support representative may take to resolve this issue

1. Update the `AIResponse` class in `model.py`.
2. Modify the system prompt in `app.py` to include this new field.
3. Test your changes with a variety of user messages.

▶ Click here for the answer

Enhancing Your Flask Application with AI Capabilities

Now that we have our AI models set up, let's integrate them into our Flask application.

Step 1: Update Your Flask Application

Let's update `app.py` to use these AI capabilities:

Open app.py in IDE

Update the content of `app.py` with:

python

```python
from flask import Flask, request, jsonify, render_template
from model import llama_response, granite_response, mistral_response
import time

app = Flask(__name__)

@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')

@app.route('/generate', methods=['POST'])
def generate():
    data = request.json
    user_message = data.get('message')
    model = data.get('model')

    if not user_message or not model:
        return jsonify({"error": "Missing message or model selection"}), 4

    system_prompt = "You are an AI assistant helping with customer inquiri

    start_time = time.time()

    try:
        if model == 'llama':
            result = llama_response(system_prompt, user_message)
        elif model == 'granite':
            result = granite_response(system_prompt, user_message)
        elif model == 'mistral':
            result = mistral_response(system_prompt, user_message)
        else:
            return jsonify({"error": "Invalid model selection"}), 400

        result['duration'] = time.time() - start_time
        return jsonify(result)
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)
```

Let's break down the changes:

1. We import our model-specific response functions.

2. In the `/generate` route, we now expect JSON input with 'message' and 'model'

3. We add error handling for missing inputs.
4. We use a try-except block to handle potential errors in AI processing.
5. We measure and include the processing time in the response.

This setup allows us to handle requests for different models and provides robust error handling.

Step 2: Create the Simple HTML file

Create the file `templates/index.html`:

Open index.html in IDE

Update the content of `templates/index.html` with:

html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI Assistant</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:ita
    <link rel="stylesheet" href="/static/styles.css">
</head>
<body class="font-ibm-plex">
    <div class="app-container">
        <!-- Header -->
        <div class="header">
            <div class="header-content">
                <h1 class="header-title">AI Assistant</h1>
                <button id="clearBtn" class="clear-btn" style="display: nc
                    <svg width="16" height="16" viewBox="0 0 24 24" fill="
                        <path d="m3 6 18 0"></path>
                        <path d="M19 6v14c0 1-1 2-2 2H7c-1 0-2-1-2-2V6"></
                        <path d="M8 6V4c0-1 1-2 2-2h4c1 0 2 1 2 2v2"></pat
                    </svg>
                    Clear Chat
                </button>
            </div>
        </div>

        <!-- Chat Container -->
        <div class="chat-container">
            <!-- Messages Area -->
            <div class="messages-area">
                <div class="messages-content">
                    <!-- Welcome Screen -->
                    <div id="welcomeScreen" class="welcome-screen">
                        <div class="welcome-icon">
                            <svg width="32" height="32" viewBox="0 0 24 24
                                <path d="M12 2C6.48 2 2 6.48 2 12s4.48 10
                                <path d="M8 14s1.5 2 4 2 4-2 4-2"/>
                                <line x1="9" y1="9" x2="9.01" y2="9"/>
                                <line x1="15" y1="9" x2="15.01" y2="9"/>
                            </svg>
                        </div>
                        <h2 class="welcome-title">Welcome to AI Assistant<
                        <p class="welcome-text">Choose a model and start a
                    </div>

                    <!-- Messages Container -->
```

```
                <!-- Loading Indicator -->
                <div id="loadingIndicator" class="loading-indicator" s
                    <div class="loading-bubble">
                        <div class="loading-content">
                            <div class="loading-dots">
                                <div class="dot"></div>
                                <div class="dot"></div>
                                <div class="dot"></div>
                            </div>
                            <span class="loading-text">AI is thinking.
                        </div>
                    </div>
                </div>

                <!-- Messages End -->
                <div id="messagesEnd"></div>
            </div>
        </div>

        <!-- Input Area -->
        <div class="input-area">
            <div class="input-content">
                <form id="chatForm" class="chat-form">
                    <!-- Model Selection -->
                    <div class="model-section">
                        <span class="model-label">Model:</span>
                        <div class="select-wrapper">
                            <select id="modelSelect" class="model-sele
                                <option value="llama">Llama</option>
                                <option value="granite">Granite</optic
                                <option value="mistral">Mistral</optic
                            </select>
                        </div>
                    </div>

                    <!-- Message Input -->
                    <div class="input-section">
                        <div class="textarea-container">
                            <textarea
                                id="messageInput"
                                class="message-textarea"
                                placeholder="Type your message... (Ent
                                rows="1"
                            ></textarea>
                        </div>

                        <button type="submit" id="sendButton" class="s
                            <svg id="sendIcon" width="16" height="16"
                                <line x1="22" y1="2" x2="11" y2="13"><
                                <polygon points="22,2 15,22 11,13 2,9
                            </svg>
```

```
                                </div>
                            </button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>

    <script src="/static/script.js"></script>
</body>
</html>
```

This is some simple HTML that will give us a form allowing us to call the `/generate` endpoint, passing a message and model selection.

Step 3: Adding CSS and JavaScript to Flask

When building a Flask web application, the HTML templates define the structure and content of your pages, but to make them visually appealing and interactive, we'll need to add CSS and JavaScript. Flask serves these static assets from a dedicated static folder. The CSS and JavaScript code for the frontend of this website is quite long, so instead of embedding it directly in the markdown (which would be messy and hard to read), we've kept things clean by hosting the files separately. To add the code, we can create a static folder in our Flask project and download the files from a GitHub Gist. Execute the below commands in the terminal one by one in order:

bash

```
mkdir static
```

Run

bash

```
wget -O static/script.js "https://gist.githubusercontent.com/tenzinmigmar/
```

Run

bash

```
wget -O static/styles.css "https://gist.githubusercontent.com/tenzinmigmar
```

Step 4: Testing Your AI-Enabled Application

First let's run our Flask application, execute:

bash

```
python app.py
```

Run

You should see output indicating that the Flask development server is running on port 5000.

The Flask application is now running locally on Cloud IDE. To access it click the following button:

Test your application

Try this with different messages and models to see how the responses vary.

Congratulations you've created your LLM-enabled Flask Application!

Conclusion and Next Steps

Congratulations on completing this guided project! You've successfully built a backend for a GenAI application using Flask, integrated multiple AI models, and implemented structured JSON outputs for enhanced functionality.

Key Takeaways

- You've learned to set up a Flask application with AI capabilities.
- You've integrated and compared multiple language models (Llama, Granite, Mistral).
- You've implemented LangChain's `JsonOutputParser` for structured AI outputs.
- You've gained insights into prompt engineering and model performance analysis.
- You've created a modular and maintainable codebase for AI integration.

Next Steps

To further enhance your skills and application:

1. **Implement Caching**: Add a caching mechanism to improve performance for repeated queries.
2. **Explore Advanced LangChain Features**: Look into features like memory for maintaining conversation context.
3. **Add More Models**: Try integrating other models available through watsonx.ai.
4. **Implement A/B Testing**: Create a system to compare responses from different models for the same query.
5. **Enhance Error Handling**: Implement more robust error handling and logging.

6. **Explore IBM Cloud Services**: Consider integrating other IBM Cloud services to expand your application's capabilities.

7. Build GenAI apps the right way with an AI coding assistant — try Bob free at ibm.biz/snp-bob.

Further Learning

- Learn about agentic AI with our hands-on learning path containing more guided projects just like this one.
- Explore the IBM watsonx.ai documentation for more advanced features.
- Dive deeper into LangChain for more sophisticated AI application architectures.
- Learn about prompt engineering techniques to improve AI model outputs.

Remember, the field of GenAI is rapidly evolving. Keep experimenting, learning, and building to stay at the forefront of this exciting technology!

Thank you for participating in this workshop. We hope you found it valuable and are inspired to continue your journey in AI-driven application development!

Enhancing Your Flask Application with AI Capabilities

Now that we have our AI models set up, let's integrate them into our Flask application.

Step 1: Update Your Flask Application

Let's update `app.py` to use these AI capabilities:

Open app.py in IDE

Update the content of `app.py` with:

python

```python
from flask import Flask, request, jsonify, render_template
from model import llama_response, granite_response, mistral_response
import time

app = Flask(__name__)

@app.route('/', methods=['GET'])
def index():
    return render_template('index.html')

@app.route('/generate', methods=['POST'])
def generate():
    data = request.json
    user_message = data.get('message')
    model = data.get('model')

    if not user_message or not model:
        return jsonify({"error": "Missing message or model selection"}), 4

    system_prompt = "You are an AI assistant helping with customer inquiri

    start_time = time.time()

    try:
        if model == 'llama':
            result = llama_response(system_prompt, user_message)
        elif model == 'granite':
            result = granite_response(system_prompt, user_message)
        elif model == 'mistral':
            result = mistral_response(system_prompt, user_message)
        else:
            return jsonify({"error": "Invalid model selection"}), 400

        result['duration'] = time.time() - start_time
        return jsonify(result)
    except Exception as e:
        return jsonify({"error": str(e)}), 500

if __name__ == '__main__':
    app.run(debug=True)
```

Let's break down the changes:

1. We import our model-specific response functions.

2. In the `/generate` route, we now expect JSON input with 'message' and 'model'

3. We add error handling for missing inputs.
4. We use a try-except block to handle potential errors in AI processing.
5. We measure and include the processing time in the response.

This setup allows us to handle requests for different models and provides robust error handling.

Step 2: Create the Simple HTML file

Create the file `templates/index.html`:

Open index.html in IDE

Update the content of `templates/index.html` with:

html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>AI Assistant</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?family=IBM+Plex+Sans:ita
    <link rel="stylesheet" href="/static/styles.css">
</head>
<body class="font-ibm-plex">
    <div class="app-container">
        <!-- Header -->
        <div class="header">
            <div class="header-content">
                <h1 class="header-title">AI Assistant</h1>
                <button id="clearBtn" class="clear-btn" style="display: nc
                    <svg width="16" height="16" viewBox="0 0 24 24" fill="
                        <path d="m3 6 18 0"></path>
                        <path d="M19 6v14c0 1-1 2-2 2H7c-1 0-2-1-2-2V6"></
                        <path d="M8 6V4c0-1 1-2 2-2h4c1 0 2 1 2 2v2"></pat
                    </svg>
                    Clear Chat
                </button>
            </div>
        </div>

        <!-- Chat Container -->
        <div class="chat-container">
            <!-- Messages Area -->
            <div class="messages-area">
                <div class="messages-content">
                    <!-- Welcome Screen -->
                    <div id="welcomeScreen" class="welcome-screen">
                        <div class="welcome-icon">
                            <svg width="32" height="32" viewBox="0 0 24 24
                                <path d="M12 2C6.48 2 2 6.48 2 12s4.48 10
                                <path d="M8 14s1.5 2 4 2 4-2 4-2"/>
                                <line x1="9" y1="9" x2="9.01" y2="9"/>
                                <line x1="15" y1="9" x2="15.01" y2="9"/>
                            </svg>
                        </div>
                        <h2 class="welcome-title">Welcome to AI Assistant<
                        <p class="welcome-text">Choose a model and start a
                    </div>

                    <!-- Messages Container -->
```

```
                <!-- Loading Indicator -->
                <div id="loadingIndicator" class="loading-indicator" s
                    <div class="loading-bubble">
                        <div class="loading-content">
                            <div class="loading-dots">
                                <div class="dot"></div>
                                <div class="dot"></div>
                                <div class="dot"></div>
                            </div>
                            <span class="loading-text">AI is thinking.
                        </div>
                    </div>
                </div>

                <!-- Messages End -->
                <div id="messagesEnd"></div>
            </div>
        </div>

        <!-- Input Area -->
        <div class="input-area">
            <div class="input-content">
                <form id="chatForm" class="chat-form">
                    <!-- Model Selection -->
                    <div class="model-section">
                        <span class="model-label">Model:</span>
                        <div class="select-wrapper">
                            <select id="modelSelect" class="model-sele
                                <option value="llama">Llama</option>
                                <option value="granite">Granite</optic
                                <option value="mistral">Mistral</optic
                            </select>
                        </div>
                    </div>

                    <!-- Message Input -->
                    <div class="input-section">
                        <div class="textarea-container">
                            <textarea
                                id="messageInput"
                                class="message-textarea"
                                placeholder="Type your message... (Ent
                                rows="1"
                            ></textarea>
                        </div>

                        <button type="submit" id="sendButton" class="s
                            <svg id="sendIcon" width="16" height="16"
                                <line x1="22" y1="2" x2="11" y2="13"><
                                <polygon points="22,2 15,22 11,13 2,9
                            </svg>
```

```
                                </div>
                            </button>
                        </div>
                    </form>
                </div>
            </div>
        </div>
    </div>

    <script src="/static/script.js"></script>
</body>
</html>
```

This is some simple HTML that will give us a form allowing us to call the `/generate` endpoint, passing a message and model selection.

Step 3: Adding CSS and JavaScript to Flask

When building a Flask web application, the HTML templates define the structure and content of your pages, but to make them visually appealing and interactive, we'll need to add CSS and JavaScript. Flask serves these static assets from a dedicated static folder. The CSS and JavaScript code for the frontend of this website is quite long, so instead of embedding it directly in the markdown (which would be messy and hard to read), we've kept things clean by hosting the files separately. To add the code, we can create a static folder in our Flask project and download the files from a GitHub Gist. Execute the below commands in the terminal one by one in order:

bash

```
mkdir static
```

Run

bash

```
wget -O static/script.js "https://gist.githubusercontent.com/tenzinmigmar/
```

Run

bash

```
wget -O static/styles.css "https://gist.githubusercontent.com/tenzinmigmar
```

Step 4: Testing Your AI-Enabled Application

First let's run our Flask application, execute:

```bash
python app.py
```

Run

You should see output indicating that the Flask development server is running on port 5000.

The Flask application is now running locally on Cloud IDE. To access it click the following button:

Test your application

Try this with different messages and models to see how the responses vary.

Congratulations you've created your LLM-enabled Flask Application!

← Setting up JSON outputs

Conclusion and Next Steps →